

TicketChain · Módulo de Productos (Eventos)

Documentación de rutas y cambios — Blockchain, Boleto NFT, PDF en IPFS y Reporte final con QR (Lab 14)

Proyectos: **mod_02_productos** (API Express + MySQL/MariaDB + Hardhat/Ganache) y **frontend_productos** (Next.js, puerto 3001). Julio 2026.

1. Visión general

El módulo gestiona **eventos/conciertos** como productos de un sistema de ticketing. Cada evento vive en MariaDB (datos mutables: nombre, descripción, banner, género, ciudad) y sus **pruebas inmutables** se replican fuera de la base de datos en tres capas complementarias:

- **Blockchain (contrato Events.sol)** — registro legible del evento on-chain (lugar, fecha, precio, aforo, organizador). Prueba de que el evento existió con esos datos y no fue alterado.
- **Boleto NFT (contrato EventoNFT.sol, ERC-721)** — el boleto como token único: se acuña (mint), se transfiere entre wallets y se verifica su dueño on-chain. Firmado por el usuario con MetaMask.
- **IPFS + PDF con QR (nuevo, Lab 14)** — el boleto/certificado del evento se genera como PDF (con QR de verificación) y se publica en IPFS, un almacenamiento descentralizado direccionado por contenido: el CID (hash) del archivo prueba que el documento no fue modificado. Además existe un **reporte final en PDF** con todos los eventos y un QR por cada uno.

Las tres capas son **best-effort**: si Ganache o el nodo IPFS están apagados, el CRUD sigue funcionando solo con la base de datos y la API nunca lanza error por ello.

Arquitectura y puertos

routes → controllers → service → MySQL (mysql2/promise). API en **http://localhost:3000**; frontend Next.js en **http://localhost:3001** (llega a la API vía rewrites de next.config.ts); Ganache en **127.0.0.1:7545** (chainId 1337); nodo IPFS local (Kubo/IPFS Desktop) con API en **127.0.0.1:5001** y gateway en ipfs.io o 127.0.0.1:8080.

2. Rutas de la API (base: /api/eventos)

2.1 CRUD de eventos (base del módulo)

Ruta	Método	Qué hace y para qué sirve
/	GET	Lista todos los eventos con todos sus campos (incluye txHash, nftTokenId, nftOwner e ipfsHash).
/:id	GET	Devuelve un evento por id. Es también la URL que codifica el QR del boleto PDF: al escanearlo se consulta la 'fuente de verdad' para contrastar lo impreso.
/	POST	Crea un evento (multipart: banner obligatorio vía multer). Genera la metadata ERC-721 en metadata/<id>.json y, si BLOCKCHAIN_ENABLED=true, registra el evento on-chain firmando con la clave del servidor.
/:id	PUT	Actualiza el evento; si llega banner nuevo borra el anterior y regenera la metadata.
/:id	DELETE	Elimina el evento y limpia sus archivos (banner y metadata).

2.2 Blockchain — contrato Events (ya existente)

Ruta	Método	Qué hace y para qué sirve
/:id/tx-hash	PUT	Flujo MetaMask: el frontend firma crearEvento() en el navegador y reporta aquí el txHash y onchainId para guardarlos en la BD como prueba on-chain. Body: { txHash, onchainId }.

La validación on-chain se hace en el frontend (lib/blockchain.ts): lee el evento del contrato con ethers (solo lectura, sin firmar) y lo compara campo a campo contra la BD.

2.3 Boleto NFT — contrato EventoNFT / ERC-721 (ya existente)

Ruta	Método	Qué hace y para qué sirve
/nft/config	GET	Devuelve address + ABI del contrato EventoNFT para que el frontend acuñe/transfiera firmando con MetaMask. Va antes de /:id en el router para que 'nft' no se interprete como id.
/nft/verify-complete/:eventoId/:address	GET	Verificación completa de solo lectura: tokenId, metadata on-chain y propiedad (¿la wallet :address es la dueña?).
/:id/nft	PUT	Persiste el tokenId y el dueño inicial tras el mint hecho en el navegador. Body: { tokenId, owner }.
/:id/nft/transfer	PUT	Persiste el nuevo dueño tras un safeTransferFrom firmado por el dueño actual. Body: { owner }.

2.4 PDF + IPFS y Reporte con QR (NUEVO — Lab 14)

Ruta	Método	Qué hace y para qué sirve
/:id/pdf	GET	Genera el PDF del boleto del evento (datos + pruebas blockchain/NFT + QR de verificación), lo guarda en pdfs/evento_<id>.pdf y lo sube al nodo IPFS local . Si IPFS responde, guarda el CID en la columna ipfs_hash y responde { filePath, ipfsHash, ipfsUrl }; si no, responde { filePath, ipfsHash:null } y el PDF queda solo local (best-effort).

Ruta	Método	Qué hace y para qué sirve
/reporte/pdf	GET	Reporte final en PDF: portada con resumen (totales, cuántos on-chain, con NFT y en IPFS) y una ficha por evento con su QR — apunta a la URL de IPFS del boleto si ya fue publicado, o al detalle en la API en su defecto. Se descarga como reporte_eventos.pdf. Declarada antes de /:id para que 'reporte' no se tome como id.

Además el servidor expone los estáticos **/uploads** (banners), **/metadata** (JSON ERC-721) y ahora **/pdfs** (boletos y reporte generados).

3. Cambios en el backend (mod_02_productos)

Dependencias nuevas: **pdfkit** (genera los PDF), **qrcode** (genera los códigos QR como imagen PNG embebida en el PDF) y **kubo-rpc-client** (cliente de la API del nodo IPFS local).

Archivo	Cambio / propósito
src/config/ipfs.js	Nuevo. Configuración de IPFS: IPFS_URL (API del nodo Kubo, por defecto 127.0.0.1:5001) e IPFS_GATEWAY (por defecto https://ipfs.io/ipfs) leídas del .env.
src/service/eventoPdfService.js	Nuevo. El corazón del Lab 14. generarBoletoPDF(evento) : arma con pdfkit el boleto (cabecera, datos del evento, sección 'Registro blockchain' con txHash/onchainId/tokenId/dueño, y un QR que codifica GET /api/eventos/:id), lo guarda en pdfs/ y lo sube a IPFS; si la subida funciona persiste el CID vía EventoService.updateIpfsHash. generarReportePDF(eventos) : reporte con resumen + ficha por evento con QR (URL IPFS o URL de la API). La subida a IPFS nunca lanza: si el nodo está apagado devuelve null y se continúa. Además de subir/pinear el archivo, lo copia al MFS del nodo (carpeta /ticketchain) con ipfs.files.cp, para que aparezca con su nombre en la pestaña Files de IPFS Desktop (ipfs.add solo pinea y la app no muestra pins sueltos).
src/service/eventoService.js	Se agregó updateIpfsHash(id, cid) : UPDATE eventos SET ipfs_hash = ? WHERE id = ?.
src/controllers/eventoController.js	Se agregaron generarPdf (GET /:id/pdf — 404 si el evento no existe, 500 con mensaje si el PDF falla) y generarReporte (GET /reporte/pdf — res.download del archivo). toResponse() ahora expone ipfsHash .
src/routes/eventoRoutes.js	Rutas nuevas: GET /reporte/pdf (antes de /:id) y GET /:id/pdf.
src/server.js	Sirve la carpeta pdfs/ como estático en /pdfs.
src/db/schema.sql	Columna nueva ipfs_hash VARCHAR(100) NULL en la tabla eventos (CID del PDF publicado). Migración para BD existentes: ALTER TABLE eventos ADD COLUMN ipfs_hash VARCHAR(100) NULL; (ya aplicada).
.env / .env.example	Variables nuevas IPFS_URL e IPFS_GATEWAY (comentadas para saber cuándo usar el gateway local 127.0.0.1:8080/ipfs).
.gitignore / pdfs/	Carpeta pdfs/ creada con .gitkeep; los PDF generados no se versionan (artefactos de runtime, igual que uploads/ y metadata/).

¿Por qué el QR del boleto apunta a la API y no a IPFS?

El QR se imprime **dentro** del PDF, y el CID de IPFS solo se conoce **después** de subir el archivo (el CID es el hash del contenido; incluirlo dentro cambiaría el contenido y por tanto el CID). Por eso el QR del boleto codifica la URL de verificación en la API (GET /api/eventos/:id), que permite contrastar los datos impresos contra la BD y, vía txHash/onchainId, contra la blockchain. El **reporte**, en cambio, se genera después de conocer los CID, así que sus QR sí apuntan directamente al PDF publicado en IPFS.

Flujo completo de GET /api/eventos/:id/pdf

1) Se busca el evento en la BD (404 si no existe). 2) qrcode genera el QR como PNG en memoria. 3) pdfkit arma el documento y lo escribe en pdfs/evento_<id>.pdf. 4) kubo-rpc-client hace ipfs.add(archivo) contra el nodo local. 5) Si respondió, se guarda el CID en eventos.ipfs_hash y se responde con la URL del gateway (https://ipfs.io/ipfs/<CID>). El mismo archivo queda accesible en http://localhost:3000/pdfs/evento_<id>.pdf.

4. Cambios en el frontend (frontend_productos)

Archivo	Cambio / propósito
lib/types.ts	El tipo Evento incluye ipfsHash (CID del PDF publicado, null si aún no se generó).
lib/api.ts	generarPdfEvento(id) : llama a GET /api/eventos/:id/pdf y tipa la respuesta (PdfResult: filePath, ipfsHash, ipfsUrl). REPORTE_PDF_URL : constante con la URL del reporte para abrirlo/descargarlo directo.
next.config.ts	Rewrite nuevo /pdfs/:path* → http://localhost:3000/pdfs/:path* para que el enlace 'Ver PDF local' funcione desde el dominio del frontend (igual que /api, /uploads y /metadata).
components/dashboard/EventsTable.tsx	Botón nuevo por evento (icono FileText , también en la vista móvil como 'PDF'): llama a generarPdfEvento y muestra un SweetAlert con el resultado — éxito con CID + enlaces 'Ver PDF en IPFS' y 'Ver PDF local', o aviso si el nodo IPFS no respondió (PDF solo local). Si el evento ya tiene ipfsHash el icono se pinta verde y el tooltip muestra el CID.
app/dashboard/page.tsx	Botón nuevo 'Reporte PDF' (icono FileDown, dorado) en la cabecera: abre /api/eventos/reporte/pdf en otra pestaña y el navegador descarga el reporte con los QR.

Botones de la tabla (columna Taquilla), de izquierda a derecha

Archivo	Cambio / propósito
Rayo (Emitir)	Solo si el evento no está on-chain: firma crearEvento() con MetaMask y guarda el txHash (PUT /:id/tx-hash).
Escudo (Verificar)	Lee el contrato Events sin firmar y compara BD vs. cadena campo a campo.
Chispas (Mint NFT)	Acuña el boleto ERC-721 con MetaMask y persiste el tokenId (PUT /:id/nft).
Insignia (Validar NFT)	GET /nft/verify-complete/...: muestra tokenId, metadata y si el boleto pertenece a tu wallet.
Avión (Transferir)	safeTransferFrom a otra wallet de Ganache; luego PUT /:id/nft/transfer con el nuevo dueño.
Documento (Generar PDF) – NUEVO	Genera el boleto PDF con QR y lo publica en IPFS (ver sección 3).
Enlace externo	Abre la metadata ERC-721 (/metadata/<id>.json).
Lápiz / Papelera	Editar (modal) y eliminar (confirmación SweetAlert).

5. Puesta en marcha y verificación

Requisitos: MariaDB/MySQL corriendo (schema src/db/schema.sql), Ganache GUI en 127.0.0.1:7545, nodo IPFS local activo (IPFS Desktop o 'ipfs daemon', API en 127.0.0.1:5001) y contratos desplegados (npm run deploy).

```
cd mod_02_productos && npm install && npm run dev # API en :3000
cd frontend_productos && npm install && npm run dev # Front en :3001
```

Pruebas rápidas sin frontend:

```
curl http://localhost:3000/api/eventos/1/pdf # → { filePath, ipfsHash, ipfsUrl }
curl -O http://localhost:3000/api/eventos/reporte/pdf # descarga el reporte con QR
```

Verificado en esta entrega: los 4 eventos existentes generan su boleto PDF y quedan publicados en IPFS (CID guardado en la BD y archivo recuperable desde el gateway local 127.0.0.1:8080), el reporte final

lista 4/4 eventos on-chain, 4/4 con NFT y 4/4 en IPFS con su QR, el frontend compila sin errores de TypeScript y los botones nuevos funcionan contra la API real.

Nota: si el nodo IPFS local está apagado, 'Generar PDF' avisa y deja el PDF accesible en /pdfs; si Ganache está apagado, el CRUD sigue funcionando solo-BD. Nada de esto rompe la API (diseño best-effort).